

Batch inference load-balancing with persistent KV-split swizzle

The Cong Luong

This technical write-up presents my solution to FlashInfer AI Kernel Generation Contest @ MLSys 2026. I initially decided to go with the Deepseek Sparse Attention [1] (DSA) track because I haven't written an attention kernel before, and this would be a very good opportunity for me to learn. To prep for this kernel, I wrote a naive implementation of flash attention [2], where we do $\text{softmax}(Q@K.T) @ V$ in a single kernel. The flaw of this naive kernel is that I partitioned the output matrix to 2D, which hurts the performance as for each `tile_N`, we have to redo $(N // \text{tile_N})$ times the computation of $Q @ K.T$. Therefore, to ensure the best performance, we should only do 1D partition for the output by tiling it along the M dimension. The concept is pretty much the same for Deepseek Sparse Attention, except that here, we are doing Flash decoding [3] instead, where we process only one token at a time, thanks to KV cache skip.

Can this kernel beat Flashinfer/TRT_LLM hand-crafted by Nvidia's kernel engineers on high throughput workloads (8x2048)? No, but it can beat it in cases where the requests per workload are imbalanced.

Check my 2 kernels DSA attention and DSA indexer and detailed explanations through lines of codes at https://github.com/luongthecong123/learn-cutedsd/tree/main/fused_kernel. As this document is only an overview of my method and couldn't capture the finer details, also in the learn-cutedsd repo, I explained the CuTeDSL [4] APIs that I used with easy, short, fully self-contained kernel so that it's easier for you and your AI agents to understand. CuTeDSL provides many quality-of-life features to make kernel development faster, while allowing you to go really low level to provide as much flexibility as possible for kernel design.

1. DSA Attention

DSA uses Multi-head latent attention, with KV of maximum length 2048. Decoding is really memory bound and doesn't benefit much from B200's high throughput tensor core engines. Here, I will try to use all the 148 SMs that B200 offers. In the detailed link I shared above, I will also explain a more over-engineered kernel with `tcgen05.mma` (UMMA), `cp.async`, `FFMA2`, warp specialization to overlap computation with memory, also a kernel with pipeline staging to overlap score and output computation, those kernels weren't in the final submission since they only improved 500 GFLOPS/s on larger workloads, it would benefit from having max num pages as input like in DSA indexer for kernel routing.

For small workloads (1,2,3,...7) I used a fully fused kernel with just warp reduction, 3 levels of reduction [5] (thread, warp reduction and thread block reduction on SMEM), these runs for just 1 wave, I set up the grid as `grid=[T, num_heads, 1]`, so each block (each SM as our register usage only allow 50% occupancy per SM) gets a unique head, so for 8 requests, we can have 128 SMs working (tensor parallelism gives each GPU 16 heads). Figure 1 shows the algorithm for this fully fused kernel, I launch the kernel with block size of 1024 threads, or 32 warps, each warp will independently calculate a column in score matrix with warp reduction, lane 0 will write results to score in SMEM. Then softmax will be applied. For output computation, due to strict precision requirement, I couldn't use tensor core with the output computation, both score and output uses FP32 for computation, loading CKV and KPE straight from GMEM without any SMEM buffering as I found this way is faster, so for output, we will also use warp, but here each warp holds 16 FP32 values in registers, for loop through N, all registers per warp will multiply with this `sm(score)` value at iter N-th with 32 warps that we have in a round robin manner.

Then each warp will store their partial sum to a SMEM buffer of shape num_warp x 512 for final reduction to get output.

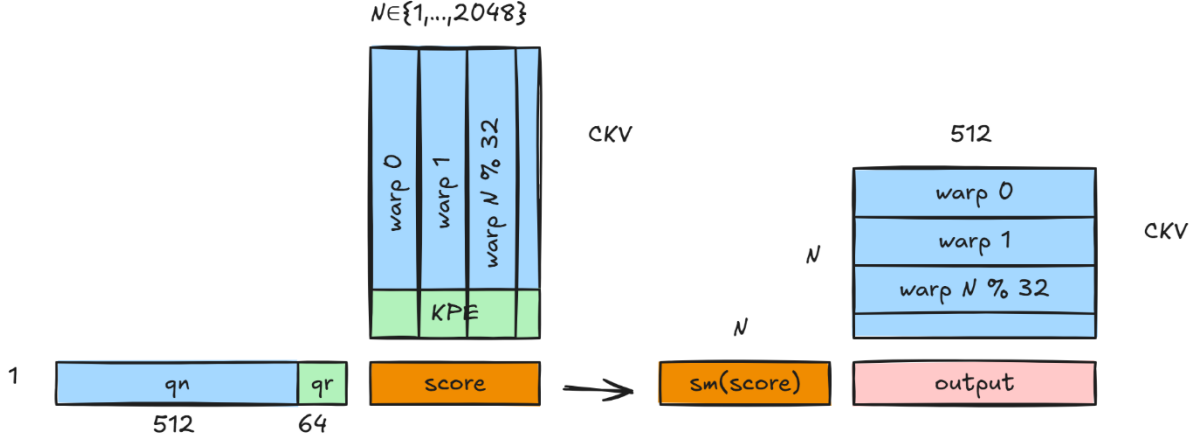


Figure 1: DSA fused kernel design

For longer requests, reaching length of 1000-2048, simply running fully fused kernel is not efficient enough, and we need to use KV split to split the work in smaller chunks, the first version I launched grid of shape $[T, \text{num_heads}, \text{num_splits}]$, a series of python script simulations shows that 8 splits with 256 tokens per split yields the best results. This kernel is followed by a reduction kernel that uses online softmax trick to combine local results from the splits. This KV split has 2 problems, the out of bound (OOB) overhead is too much and the work assigned to each SM is extremely imbalanced, intra-kernel profiling described in this blog post [6] helped spotting this problem during my kernel development. The first problem can be solved with persistent kernel, a grid of shape $[\text{num_heads}, \text{num_splits}]$ is launched instead, that's 128 SMs, meaning we have 20 SMs left, those will be used to launch the reduction kernel at the same time of the DSA kernel with the help of Programmatic Dependent Launch (PDL) to hide launch and independent epilogue overhead, more details in my learn-cuteds1 repo.

KV-split workload distribution (split size = 256 tokens)

	block 0	block 1	block 2	block 3	block 4	block 5	block 6	block 7
request 0 (seq=288)	256	32						
request 1 (seq=4)								
request 2 (seq=1884)	256	256	256	256	256	256	256	92
request 3 (seq=21)	21							
request 4 (seq=136)	136							
request 5 (seq=2048)	256	256	256	256	256	256	256	256
request 6 (seq=42)	42							
request 7 (seq=335)	256	79						

Figure 2: Workload imbalance in batch inference

This persistent kernel pattern makes each block calculate a split in all requests. This pattern creates imbalance. From Figure 2, we can see that block 0 has to process 8 splits while the block 2 – 7 only need to process 2 splits, assuming OOB splits have zero latency. To relieve pressure on block 0, swizzle XOR is used. Where block 0 will process $\text{split_idx_new} = \text{XOR}(\text{split_idx_old}, \text{request_idx})$, resulted in splits sitting on the diagonal line, with XOR swizzle, block 0 now only has to compute 3 splits instead of 8.

The prologue has 2 tasks: for each request, it finds the sequence length for early exit and OOB split detection and stores the sparse indices into SMEM for faster access. Where sparse indices point to specific KV cache location created by the following kernel. My 2 kernels didn't exploit page size = 64, and hence, it can work for page size = 1 too.

2. DSA Indexer

This kernel also benefits from swizzle XOR to balance the workload, but we only have 1 request per workload to use indexer, the remaining seq len < 2048, will be pass through, no computation done, just pure memory ops. The hardest part of this kernel is how to use cp.async to copy GMEM to the swizzled layout in SMEM that tensor core needs. We need to load data from GMEM into the layout in SMEM that UMMA needs, for this I used cp.async with tiled TV layout for more bytes in flight loads, each warp will load a single row from GMEM into the special layout in SMEM, CuTeDSL provides tgen05 layouts out of the box to save us time reading PTX documentation. Figure 3 shows the Deepgemm FP8 q and FP32 scaling factor format, they are both contiguous which allows easier loading with cp.async.

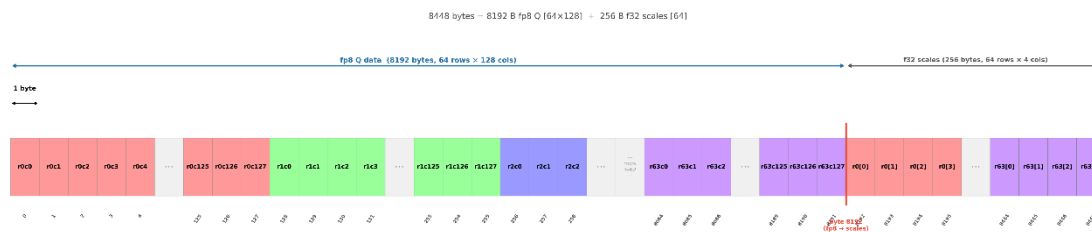


Figure 3: F8 q and F32 scaling factor packing pattern

Following KV split patterns, dim_split here will be of shape 128 to fit SM100 tensor core M dimension of 128, which is faster than 64, as empirically shown in [8], also, it makes TMEM -> RMEM loading easier and straightforward. The difference is that, here DSA indexer uses ReLU instead of Softmax so we don't need online softmax to stitch things together. There are a few kernels, a pass-through kernel which directly copies the indices if sequence length < 2048 (or num pages < 32). We will launch a grid of [num_splits + num_requests, 1, 1], the first num_splits blocks will be launched first and uses persistent pattern to loop through requests with > 2048 sequence length, the last num_requests blocks will handle cases with num pages < 32. Launching compute heavier workloads first keep them alive on a SM, while the rest of the pass-through blocks can run on idle SMs, although in this contest, all workloads happen in just 1 wave. PDL is also used here to help with topk kernel launch overhead.

Figure 4 shows the fully fused kernel design, here we will perform $B @ A.T = C.T$ to bring the dimension of K to the M dimension of UMMA, so we can use 128m64n32k FP8 tensor core instruction on B200. The cp.async loading to SMEM used 512 threads for more bytes in flight, while the epilogue that loads mma result from TMEM to RMEM uses only a warp group of 128 threads, each thread holds 64 F32 values in their registers (25% SM occupancy), and all other ops like F32 scaling, ReLU and weighted sum over heads is done directly on RMEM to avoid round trips back to GMEM.

For the topk, I used histogram + radix sort to speed it up, basically, with bin size = 256, you just need that last 8 bins. Most of the overhead is spent on topk which accounts for 80% of the indexer latency, while the indexer with FP8 tensor core is much faster. So for future optimizations, more attention should be spent on the topk kernel.

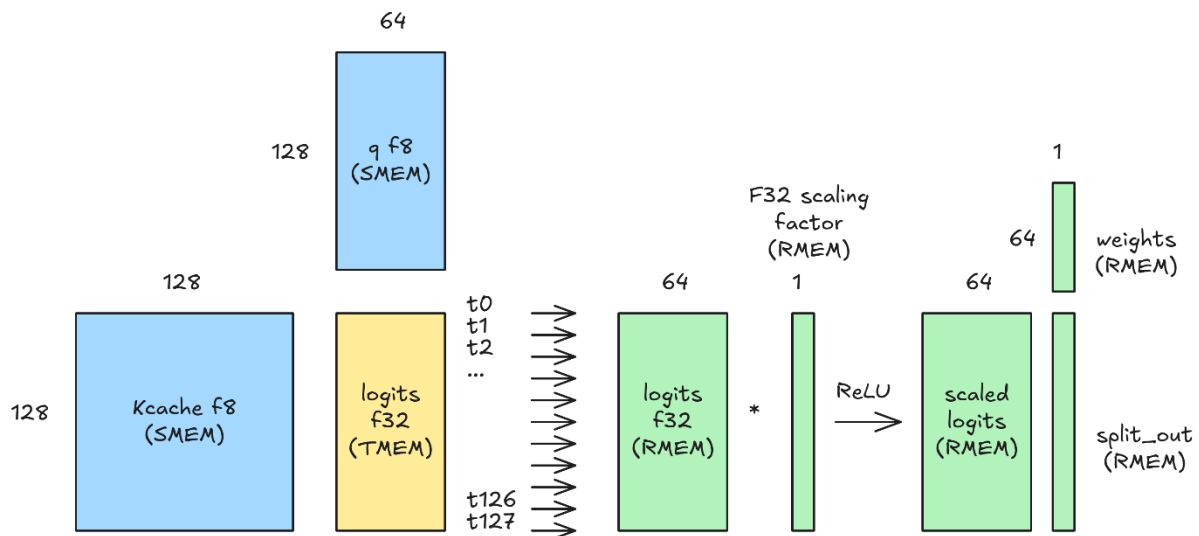


Figure 4: DSA Indexer fused kernel design

3. Agentic coding

Normally, writing kernel with a team of experienced kernel engineers to explore different ideas, different experiments. Now, with agents, a single engineer can craft a 90-95% performance of the team-designed kernel in the same amount of time. The engineer will first design the overall fused kernel, draft a first version by hand, then assign each step to a different agent for gradual optimization of this draft version, for even more speedup, then agent's ideas are not enough and requires human to give ideas or find ideas online and give it for it to implement.

Human start with a first implementation that is runnable, shows them how to run it, it passed correctness with a base line duration (verifiable results). Intra kernel profiling can be really useful as we know the latency for each step in the kernel and set that as a target for agents to optimize further, it's also important to create minimal version for each step in a separate and isolated file so each agent doesn't conflict with each other.

They can be good at optimizing this early draft kernel, they did decent speedups for the softmax and well for the output computation phase. Once reached a point, it's really hard for them to speed things up further and need human guidance to use things like vectorized loads. Or for humans to show them great ideas to experiment.

They can easily give up and not explore further and don't see the potential of a method if this experiment with this method yields slower speedup. One example is the persistent XOR KV-split kernel, they couldn't implement it to the ideas that I explained, I had to spend time writing that entire kernel by hand. They also did great with optimizing top-k kernel. Also, they seem to not able to search the internet and look at well optimized kernels and use ideas from those to adapt and experiment, instead they try to look for ideas already in the code base, from their KV cache or from their own weights.

Additionally, I spent 1 and a half month working on these 2 kernels on my free time, I subscribed to Copilot Pro + which is 40\$ per month, apart from included credits, I also spent around 200 \$ on extra credits, totaling 280\$ on agents, a large sum contributed to the expensive Claude Opus, despite lesser usage than Claude Sonnet. For B200 spendings on Modal for kernel development, I used around 100\$ worth of credit.

References

1. DeepSeek-AI. "DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models." arXiv:2512.02556, 2025. <https://arxiv.org/abs/2512.02556>
2. Dao, Tri, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness." arXiv:2205.14135, 2022. <https://arxiv.org/abs/2205.14135>
3. Dao, Tri, Daniel Haziza, Francisco Massa, and Grigory Sizov. "Flash-Decoding for long-context inference." PyTorch Blog, 2023. <https://pytorch.org/blog/flash-decoding/>
4. NVIDIA. "CuTe DSL Introduction." NVIDIA CUTLASS Documentation, 2025-2026. https://docs.nvidia.com/cutlass/latest/media/docs/pythonDSL/cute_dsl_general/dsl_introduction.html
5. Guo, Wentao, Ted Zadouri, and Tri Dao. "Getting Memory-bound Kernels to Speed-of-Light." Dao-AILab QuACK, 2025. <https://github.com/Dao-AILab/quack/blob/main/media/2025-07-10-membound-sol.md>
6. <https://gau-nernst.github.io/tcgen05/>
7. <https://yang-yifan.github.io/blogs/pdl/pdl.html>
8. <https://newsletter.semianalysis.com/p/dissecting-nvidia-blackwell-tensor>